Comparative Analysis of Jakarta Composite Index (JCI) Forecasting Using Traditional and Machine Learning Time Series Approaches

Marcell Prasetyo
BINUS University
Jakarta, Indonesia

Aaron Jeremy Alexander
BINUS University
Jakarta, Indonesia

Abstract - **Predicting stock market prices can help make better investment decisions and also reduce risk. This study compares six forecasting approaches to predict the daily closing price of the Jakarta Composite Index (JCI), starting with Naive, Naive with Drift, Simple Exponential Smoothing (SES), Double Exponential Smoothing (DES), ARIMA, and Artificial Neural Networks (ANN). The data was collected from January 1, 2024, to May 13, 2026, and covered 558 trading days. To make sure of equal comparisons, all the models used the same training and testing data. For the ANN model, six random seeds were used. The results were considered average. Model performance was evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE). The results showed that the ANN model performed best, with a MAPE of 0.6149%, while ARIMA(2, 1, 2) performed the worst, with a MAPE of 1.963%.**

Keywords: **Jakarta Composite Index (JCI), Stock Market Forecasting, Artificial Neural Network (ANN)**

1. INTRODUCTION

1.1. Background

Many people use the Jakarta Composite Index, also called the JCI, to monitor the performance of the Indonesian stock market [1]. Predictions about the development of the JCI are very important, especially to investors, fund managers, and policymakers because they have the ability to help them make better decisions about finances and managing risks [2]. However, many factors influence the market, such as economic conditions, foreign capital flows, and global events [2]. These factors often cause stock prices to fluctuate, which makes predictions less accurate [4].

Financial time series forecasting can be done in different ways [5]. Conventional methods like Naive and Naive with Drift use historical data to predict future prices [3]. Simple Exponential Smoothing (SES), Double Exponential Smoothing (DES), and ARIMA also use statistics to find patterns in past data [6]. Although these models are easy to implement, thus making them widely used, they may not be that effective when market conditions change rapidly [1].

To solve this problem, researchers are increasingly using artificial neural networks (ANNs) for time series forecasting [5]. ANNs have the ability to learn complex patterns that reside in data without making many assumptions [5]. However, this does not mean that they always perform better than conventional methods. Well-tuned statistical models can outperform ANNs in some situations [5].

1.2. Problem Foundation

a. How does the performance of conventional time series models compare to machine learning models in predicting JCI?
b. How much do sudden market changes and high volatility affect the forecasting accuracy of traditional linear models?
c. Do machine learning models perform better than traditional forecasting methods in predicting out-of-sample JCI movements?

1.3. Objectives

a. To evaluate and compare the forecasting performance of traditional statistical models and machine learning models using JCI data from 2024 to 2026
b. To examine the impact of market volatility and sudden market changes on the forecasting accuracy of traditional linear models
c. To determine whether machine learning models perform better than traditional methods when forecasting unseen JCI data

1.4. Benefits

This research finds out how different methods in forecasting perform with stock (JCI) data. Starting from the most standard and basic models, which are Naive, Naive with Drift, and Exponential Smoothing. Then it continues to more advanced models such as ARIMA and ANN. This contrast will help determine which method is

the most effective when trying to predict future values of JCI under different market conditions. The results gathered from this research might also provide some comprehension into the predictability of the Indonesian stock market during different periods of fluctuation

2. THEORETICAL STUDY

2.1. Basic Naive Model

The most recent value can be predicted by the Basic Naive Model [3]. For stock market data, this means that the last observed closing price is the same as the predicted closing price for the next day. This model is usually used as a benchmark to compare it with other forecasting methods because of its simple structure. The model is written below:

$$Y_{t+1} = Y_t$$

Since it does not require that much preparation and computation of the data, this model is easy to use, often making it a standard when comparing other forecasting models. Despite its simplicity, this model carries some limitations since it can only predict recent data, making it unable to explain trends or rapid changes in the market. Therefore, when the prices rapidly change, the predicted values might not reflect actual movements in the market [3].

2.2. Naive Model with Drift

The Naive Model with Drift is a variation of the basic Naive model that accounts for trends within the data [3]. This model changes the forecast by using the average historical differences rather than making assumptions that the future values will not change. This gives the model the ability to show the overall direction of the data and makes it acceptable for long-term trends in time series. The model is written below:

$$Y_{t+h} = Y_t + \frac{h}{t-1}\sum_{i=2}^{t}(Y_i - Y_{i-1})$$

2.3. Simple Exponential Smoothing (SES)

The Simple Exponential Smoothing method is used for time series data that do not have a strong trend in them. SES uses past data to compute predictions. The more recent the values are, the more they are weighted.

Increasing the alpha value makes the model respond faster to recent changes, while lowering the alpha value results in smoother forecasts. The model is written below:

$$\hat{Y}_{t+1|t} = \alpha Y_t + (1-\alpha)\hat{Y}_{t|t-1}$$

2.4. Double Exponential Smoothing (DES)

For time series data that have a trend, the Double Exponential Smoothing model creates an estimation of the current level and the trend. In this method, the two parameters are alpha and beta, where alpha controls the level, while beta controls the trend. This allows the method to create forecasts that mirror changes that are present in the data as a collective. The model is written below:

$$L_t = \alpha Y_t + (1-\alpha)(L_{t-1} + T_{t-1})$$

$$T_t = \beta(L_t - L_{t-1}) + (1-\beta)T_{t-1}$$

2.5. Autoregressive Integrated Moving Average (ARIMA)

In ARIMA, there are three parameters, which are p, d, and q. Parameter q is the moving average component, which supports future forecasts by using past forecast errors. [6]. Next up is the parameter d, which represents the order of differentiation, which calculates the difference between consecutive observations, helping stabilize the data [6]. Together with the last parameter p, which represents the autoregressive part, ARIMA allows for making models of past patterns and expecting future values. The model is written below:

$$Y_t' = c + \phi_1 Y_{t-1}' + \cdots + \phi_p Y_{t-p}' + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q} + \epsilon_t$$

2.6. Artificial Neural Networks (ANN)

Artificial neural networks, also known as ANNs, are machine learning models commonly used for prediction and forecasting [5]. They have the ability to document complex data patterns without making strict assumptions about the underlying relationships between variables.

ANNs consist of an input layer, hidden layers, and an output layer [5]. To improve model performance, the network adjusts its weights according to prediction errors during training as data passes through these layers [5].

3. METHODOLOGY

3.1. Research Design

In this research, the quantitative methods were used to make forecasts of the daily closing price within the JCI. To evaluate the performances and results, R and different forecasting techniques were compared [3][7]

Basic Naive, Naive with Drift, Simple Exponential Smoothing (SES), Double Exponential Smoothing (DES), ARIMA, and Artificial Neural Networks were among the models that were used in this

research [5]. To ensure consistent and fair comparisons, all models used the same training and testing data.

3.2. Data Environment and R Packages

In this study, R was used for all data processing, model evaluation, and forecasting tasks [3]. The following libraries was used:

- Data Management and Math**:** `Pandas` is utilized for data frame manipulation and lag generation, while `NumPy` provides supporting mathematical operations [9].

- Statistical Modeling and Testing**:** The forecast package was used to implement the Naive, Naive with Drift, SES, and DES models [4]. The tseries package was used for the Augmented Dickey Fuller (ADF) test, while car and lmtest were used for data transformation and regression analysis [6]. Residual normality was evaluated using the NORTEST package [12].

- Neural Network Modeling: The NeuralNet and NNet packages were used to develop and train the artificial neural network models [13].

- Performance Evaluation: Forecasting performance was measured using functions provided by the Metrics package [14].

3.3. Data Collection and Target Variable

The data used in this research comes from the "*Data Historis Jakarta Stock Exchange.csv*" file. This file contains trading data for the Jakarta Composite Index (JCI) from the past [2]. This data can be accessed and acquired from Investing.com.

The data starts from January 1st, 2024, to May 13th, 2026. The dataset only contains 558 rows of data (days), even though this period spans 864 calendar days. This is due to the fact that the stock market is closed on weekends, national holidays, and other non-trading days.

The variables in this dataset are Tanggal (trading date), Terakhir (closing price), Pembukaan (opening price), Tertinggi (high price), Terendah (low price), Volume, and Change% (perubahan%). In this research, the main variable that will be used is Terakhir (closing price). This data is used to evaluate the forecasting model because it covers a wide range of market conditions during the observation period.

3.4. Data Pre-Processing and Transformation Pipeline

Before model development, the dataset was cleaned and prepared in R. The following preprocessing steps were performed:

- Syntax and Data Cleaning

The read.csv() function is used to import the raw data file into R. The values contained in the data are first cleaned before analysis because data acquired from the internet may contain punctuation marks used as thousand separators. Period symbols can be removed using the gsub() function, which changes commas to decimal points. Next, the as.numeric() function is used to convert the cleaned values to numeric format [9].

- Target Variable Identification

The target variable for this study is the daily closing price, referred to in the dataset as "Terakhir." This variable represents the stock index value at the end of each trading day.

- Data Reversing and Sequential Sorting

In most cases, historical stock data downloaded from the internet is arranged with the most recent dates appearing first. To maintain chronological order, the rows in the data set are reversed so that the oldest observation begins and the most recent ends.

- Sequential Data Splitting

Using an 80:20 split, the dataset was divided into training and testing sets. The first 446 observations were used as training data (train_data), and the last 112 observations were used as testing data (test_data). The training data was used to build the forecasting model, and the testing data was used to evaluate its performance.

- Variance and Stationary Stabilization

Before using the model for forecasting, a number of tests are performed to evaluate the training data, mainly the test to evaluate whether the training data is stationary towards both the mean and variance or not. The summary(powerTransform(train_data) function is used to check the data distribution. The log() function is used to apply a logarithmic transformation to improve the stability of the data if the data is not stationary towards the variance. The Augmented Dickey Fuller (ADF) test is then performed with adf.test() to evaluate if the data is stationary towards the mean or not. By using diff(transformed) and the logarithmically transformed data, this test can be applied.

3.5. Model Implementation

- Basic Naive Model

By using the naive() function and h = n_test, the basic naive model uses the last

observed value of the training data as a forecast for all observations in the test period. This makes it easy to compare with other forecasting models [3].

- Naive Model with Drift

This model is run using the random walk function rwf() with the argument drift = TRUE. It calculates the average change between the first and last days of the training set to project a steady, sloped trend line across the testing steps [3].

- Simple Exponential Smoothing (SES)

This system is built using a looping code that evaluates multiple smoothing parameters, alpha (α), across the training data. It calls the ets() function with the model configuration argument set to "ANN" (representing additive error, no trend, and no seasonality) to isolate the single best weight for short-term index forecasts [4].

- Double Exponential Smoothing (DES)

This model is implemented by executing the ets() function with the structural arguments set to "AAN" (representing Additive Error, Additive Trend, and No Seasonality). A looping code automatically tests a selection of trend-smoothing parameters beta (β) alongside a level-smoothing parameter alpha (α) to evaluate how effectively the framework can track the moving trend line of the stock index [4].

- ARIMA

To find the optimal model structure, the code reviews the partial autocorrelation (PACF) and autocorrelation (ACF) graphs of the data. A loop tests multiple combinations of past-price orders (*p)*, differencing steps (*d)*, and error-correction moving average orders *d* using the arima() function [6]. Residual diagnostics are verified using the Ljung-Box test via Box.test(..., type = "Ljung-Box") and the Lilliefors normality test via lillie.test() [12]. Final training and testing forecasts are back-transformed out of the log scale using the exponential function exp().

- Artificial Neural Networks (ANN)

This machine learning framework is built using the neuralnet() function. The data layout is structured by a looping code that automatically generates a lagged variable data frame where the historical lag column acts as the primary input feature to discover non-linear patterns [5]. To keep the matrix calculations stable, the entire data frame is scaled down into a range between 0 and 1 using a custom normalization function:

$$\text{Normalize}(x) = \frac{x - \min(x)}{\max(x) - \min(x)}$$

To make sure the results are stable and fair, the network does not rely on just one single test. Instead, the loop automatically runs the model 6 separate times using 6 different random seeds. Each run uses a hidden node layer configuration and runs linearly. output = TRUE over a maximum threshold of 1e6 steps. After generating predictions using the predict() function, the scaled output values are converted back to their original market scale using the saved original minimum and maximum values [13]:

$$\text{Price} = \text{Predicted_Scaled} \times (\max(y) - \min(y)) + \min(y)$$

After all 6 runs with different random seeds were completed, the error metrics from each run were calculated and averaged. The average error across multiple runs was used to evaluate model performance. This helps reduce the impact of random initialization on the final results.

3.6. Evaluation Metrics

To evaluate which model performs best for JCI forecasting, the predicted values are compared with the actual values in the testing dataset. Three performance metrics from the Metrics package are used to measure the forecasting error of each model [14]:

- Mean Absolute Error (MAE): Calculated using the mae() function, MAE measures the average absolute difference between the predicted and actual values. It indicates how far the model's predictions deviate from the actual values on average [10].

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- Root Mean Squared Error (RMSE): Calculated using the rmse() function, RMSE measures the square root of the average squared differences between the predicted and actual values. Since the errors are squared before averaging, RMSE gives greater weight to larger prediction errors [15].

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- Mean Absolute Percentage Error (MAPE): Calculated using the mape() function, MAPE measures the average prediction error as a percentage of the actual values. This metric provides an easy way to evaluate and compare the accuracy of different forecasting models.

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100\%$$

4. RESULT AND DISCUSSION

4.1. Time Series Pattern Analysis

A visual analysis of the time series plot for the data is done to identify the pattern. Figure 1 shows the time series plot of the data.

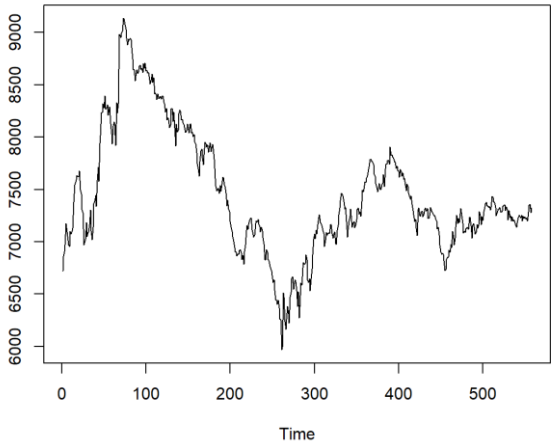


Figure 1. Time Series Plot of 'Close'

Figure 1 shows a time series plot of the 'close' variable of JKSE. Initially, the series shows a strong upwards trend from time 0 to around 80 where it rises from around 6700 to 9000. Afterwards, it shows a steady decline to around time 260 where it drops to around 6000. Following the decline, it recovers and increases and stabilizes at around 7200 nearing the end.

The patterns show a combination of upwards and downwards trend along the periods. In addition, it exhibits cyclic patterns as seen in the upwards and downwards trend that occur in non-fixed repeating intervals. There is no clear seasonal pattern because of that. Furthermore, the data shows signs of non-stationarity due to the volatile changes across the observation period.

The observed pattern also suggests that the data exhibits random walk characteristics, as seen in most financial time series data [17]. The changes are volatile and unpredictable. This also suggests that future observations are heavily influenced by more recent values.

4.2. Stationarity Testing

Stationarity testing is done using the training data to identify whether the data satisfies ARIMA assumptions for modeling. First, a power transform test is done to identify whether the data is stationary towards the variance and whether or not a transformation is needed. The hypothesis for this test is as follows:

H0: lambda = 1

H1: lambda =/= 1

The resulting p-value for this test is 0.001. Using an alpha value of 0.05, the p-value is less than alpha. Therefore, H0 is rejected and it can be concluded that the data is not stationary towards the variance and a transformation is needed to be done.

The next test is to test data for its stationarity towards the mean. The test is conducted using an Augmented-Dickey Fuller Test (ADF test). The hypothesis for this test is as follows:

H0: Not stationary towards the mean
H1: Stationary towards the mean

The resulting p-value for this test is 0.418. Using an alpha value of 0.05, the p-value is more than alpha. Therefore, H0 has failed to be rejected and it can be concluded that the data is not stationary towards the mean.

The data failed both tests. This means that the data is non-stationary. This aligns with the visual analysis of the data and supports the trend-cyclic like pattern of the data. In addition, the non-stationarity of the data is consistent with the possible behavior of a random-walk pattern in financial time series data.

A natural log transformation is done to the training data. The newly transformed data will be tested again with the same tests using the same hypothesis.

In the case of the power transform test, the resulting p-value for the test is 0.2023. Using the same alpha value, the p-value is more than alpha. Therefore, H0 has failed to be rejected and it can be concluded that the transformation is proper and the data is now stationary towards the variance.

In the case of the ADF test, the resulting p-value for the test is 0.439. Using the same alpha value, the p-value is more than alpha. Therefore, H0 has failed to be rejected and it can be concluded that the transformed data is not stationary towards the mean.

The transformed data fails the ADF test which means that the data is still not stationary. To solve this problem, differencing will be done to the transformed data and an ADF test will be conducted to check its stationarity.

Using the same hypothesis and alpha value, the resulting p-value is 0.01 which is less than alpha. Therefore, H0 is rejected and it can be concluded that the differenced transformed data is stationary towards the mean. This results in the differenced, transformed data is now stationary and satisfying ARIMA assumptions for modelling later. In conclusion, the ARIMA models will use the transformed data with the 'd' parameter valuing at 1 since differencing is done once.

4.3. Naive Models

The naive models are evaluated to be the baseline forecasting model for comparison with the other advanced models. The basic Naive method predicts future observations with the most recent observation. In addition, a drift variation of the naive model is also tested to account for the trend component within the data. Naive with Table I records the performance of each model.

TABLE I. Naive Models Performance

Model	RMSE	MAE	MAPE (%)
Basic Naive	163.831	138.889	1.937
Naive with Drift h = 112	135.308	103.035	1.446
Naive with Drift h = 1	43.500	43.500	0.594
Naive with Drift h = 7	85.368	71.753	0.980
Naive with Drift h = 14	48.298	38.835	0.533
Naive with Drift h = 30	124.2352	114.552	1.587
Naive with Drift h = 60	135.308	103.035	1.446

From the table, Naive with Drift performs better than Basic Naive using the same forecasting period. It has the lowest RMSE, MAE and MAPE values when compared to Basic Naive. This result is expected as Naive with Drift is able to capture random-walk patterns better than Basic Naive. Therefore, the Naive with Drift model will be used as a benchmark for other forecasting models [16].

For shorter forecasting periods, Naive with Drift performs significantly better. In terms of RMSE, the forecasting period of 1 achieved the lowest RMSE value. Meanwhile, the forecasting period of 14 achieved the lowest MAE and MAPE values. Naive with Drift naturally performs better in shorter forecasting periods as future observations of random walk data are mostly influenced by more recent observations. Overall, the best Naive model is Naive with Drift with a forecasting period of 14 as it achieved the lowest MAE and MAPE values while having a decent enough RMSE value.

To account for the fairness during model comparison later, Naive with Drift using a forecasting period of 112, which is the length of the test set, will be used as the benchmark model. Those models should aim to outperform the benchmark model.

4.4. Exponential Smoothing Models

Two types of exponential smoothing models are done which are Simple Exponential Smoothing and Double Exponential Smoothing. Each model is tested with 5 different parameters each. The goal is to compare the two ES models and determine which model performs best on the data.

4.4.1. Simple Exponential Smoothing (SES)

Five different SES models with different alpha parameters are tested and compared to each other. Table II records the performance of each model.

TABLE II. SES Models Performance

Alpha Value	RMSE	MAE	MAPE (%)
0.9	161.340	135.335	1.889
0.7	156.792	128.467	1.797
0.5	153.575	121.321	1.703
0.3	154.408	115.498	1.629
0.1	169.958	120.100	1.704

From the table, SES(0.5) achieved the lowest RMSE value of 153.575 which indicates the best overall forecasting performance in terms of minimizing large prediction errors. This suggests that SES(0.5) provides the most stable forecasting results as RMSE penalizes large errors more heavily.

Meanwhile, SES(0.3) achieved the lowest MAE and MAPE with values of 115.498 & 1.629% respectively. This suggests that the model generates smaller average forecasting deviations and lower percentage errors compared to other models. Other than that, extremely high or low alpha values results in worse performance for the SES model.

Overall, the most optimal choice for SES is SES(0.3) as it has the lowest MAE & MAPE value while its RMSE value does not differ greatly from SES(0.5).

4.4.2. Double Exponential Smoothing (DES)

DES introduces a second parameter which is beta. Multiple combinations of alpha and beta parameter values will be used. Ten different DES models will be tested and compared. Table III records the performance of each model.

TABLE III. DES Models Performance

Alpha	Beta	RMSE	MAE	MAPE (%)
0.3	0.01	155.104	119.887	1.686
0.1	0.05	156.138	118.946	1.675
0.2	0.1	165.829	139.984	1.954
0.3	0.2	204.533	184.777	2.563
0.4	0.3	233.747	213.704	2.958
0.5	0.4	252.097	230.975	3.194
0.6	0.5	274.779	251.813	3.480
0.7	0.3	230.690	210.623	2.915
0.8	0.1	181.120	159.166	2.212
0.9	0.7	321.367	294.667	4.069

From the results, DES(0.3, 0.01) achieved the lowest RMSE value of 155.104. Meanwhile, DES(0.1, 0.05) achieved the lowest MAE and MAPE values of 118.946 and 1.675% respectively. Combinations of high

alpha and beta parameter values yield worse performance for the DES model. On the other hand, combinations of low alpha and beta parameter values yield better performance for the model. Models with low beta values perform better than those with high beta values. The same could be said for the alpha values. Overall, the best model for DES is DES(0.1, 0.05) as the achieved the lowest MAE and MAPE values while having a RMSE value not far from DES(0.3, 0.01).

Compared to the SES(0.3) model, DES(0.1, 0.05) does not improve on the forecasting accuracy. This indicates that the trend component may not follow a smooth linear pattern which limits the performance of trend-based smoothing models like DES. In conclusion, SES(0.3) is the most optimal exponential smoothing model.

4.5. ARIMA Models

4.5.1. ARIMA(p, d, q) Orders

To identify ARIMA orders, specifically 'p' and 'q', ACF and PACF plots are used. The differenced and transformed data will be used to create the plots.

The ACF plot is used to identify the MA(q) order. Figure 2 shows the ACF plot for the data.

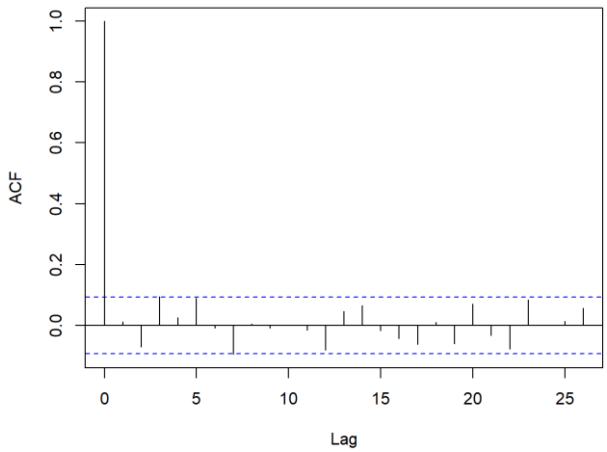


Figure 2. ACF Plot

From the ACF plot, there seems to be no significant spikes from the lags. Lags 3 and 7 seem to barely pass the boundary line. This suggests that the data may have a short-term dependency, although very weak one, on lag 3 and lag 7. Another possibility is that they are only minor and random fluctuations that don't reflect the true pattern. There is also no clear cutoff in the lags. In this case, the order would be 0. For further analysis, those three values will be used.

Meanwhile, the PACF plot is used to identify AR(p) order. Figure 3 shows the PACF plot of the data.

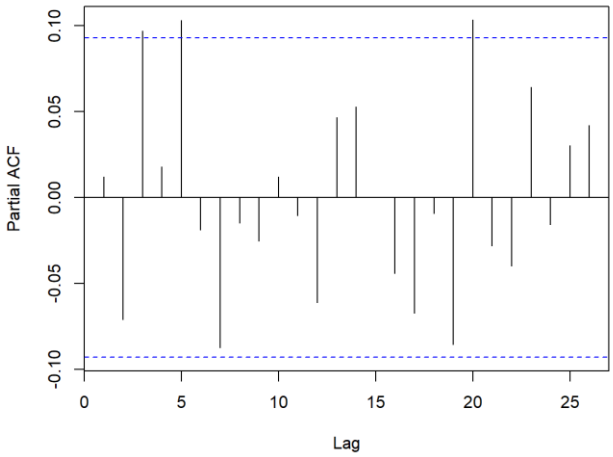


Figure 3. PACF Plot

The plot shows several significant spikes on lags 3, 5 and 20. However, there is no clear cutoff that could suggest the AR structure of the model. Most spikes remain under the boundary line and relatively small. An autoregressive process of order p would have a cutoff after lag p [18]. Since there is no clear cutoff after a certain lag, the order for the autoregressive process is 0 with either 3 or 5 having a small influence.

4.5.2. Model Fitting

Different models are fitted using different ARIMA orders. In total, there are 10 different models that are tested and compared. The models chosen have significant parameters. Table V records the performance of each model.

TABLE V. ARIMA Models Performance

Model	RMSE	MAE	MAPE (%)
ARIMA(2,1,2)	165.317	140.875	1.963
ARIMA(3,1,2)	170.560	147.491	2.053
ARIMA(0,2,1)	316.995	285.315	3.950
ARIMA(1,2,0)	2799.906	2473.318	34.137
ARIMA(2,2,0)	2511.106	2206.046	30.442
ARIMA(3,2,0)	2242.911	1960.842	27.053
ARIMA(1,3,1)	3276.038	2868.408	39.585
ARIMA(0,3,1)	3757.645	3327.606	45.938
ARIMA(1,3,0)	6430.569	6068.570	83.985
ARIMA(2,3,1)	2974.146	2586.850	35.692

From the table, ARIMA(2,1,2) achieved the best performance out of the other ARIMA models as it achieved the lowest RMSE, MAE and MAPE values of 165.317, 140.875 and 1.963% respectively. Models with higher differencing orders more than 1 perform significantly worse than models with its differencing order at 1. This is because from the previous ADF test, it is shown that differencing is only needed to be done once. Therefore, the best model for ARIMA is ARIMA(2,1,2).

4.5.3. Diagnosis Checking

To confirm the validity of ARIMA(2,1,2). Diagnosis checks are done. These checks aim to evaluate whether the model is adequate enough to be used.

4.5.3.1. Parameter Hypothesis Testing

The parameter estimates of ARIMA(2,1,2) will be tested to see the significance of each parameter of the model. The alpha value for this test is 0.05. The hypothesis for this test is as follows:

H0: bi = 0
H1: bi =/= 0

Table VI records the coefficient estimates, p-values for each parameter and the decision of the rejection of H0.

TABLE VI. Parameter Hypothesis Testing

Parameter	Estimate	P-value	Reject H0?
AR(1)	-1.559657	2.2x10-16	Reject
AR(2)	-0.971015	2.2x10-16	Reject
MA(1)	1.601662	2.2x10-16	Reject
MA(2)	0.984406	2.2x10-16	Reject

From the table, every parameter for ARIMA(2,1,2) rejects H0 as their p-value is less than alpha. Therefore, it can be said that the AR and MA components of the model are significant to the model.

4.5.3.2. Normality & White Noise of Residuals

Next, the normality and white noise of residuals are tested. The normality test is done using Lilliefors test. The hypothesis for this test is as follows:

H0: The residuals are normally distributed.
H1: The residuals are not normally distributed.

The p-value for this test is 9.23x10-8. Using an alpha value of 0.05, the p-value is less than alpha. Therefore, H0 is rejected and it can be concluded that the residuals are not normally distributed.

The next test is to test whether the residuals are white noise using the Ljung Box test. The hypothesis for this test is as follows:

H0: The residuals are white noise.
H1: The residuals are not white noise.

The p-value for this test is 0.7885. Using an alpha value of 0.05, the p-value is more than alpha. Therefore, H0 has failed to be rejected and it can be concluded that the residuals are white noise.

From the results of the test, the model passes the white noise test while failing the normality test. This is caused by the random nature of stock. As shown in Figure 1, the values can fluctuate unpredictably which can cause residuals to be non-normal. The ARIMA model is not valid but still will be used for comparison later.

4.6. ANN Models

ANN models are implemented to capture the non-linear nature of the data. Unlike traditional models, ANN models do not rely on strict linear assumptions that those models are bound to [19]. The ANN models accept inputs from the input layer in which the result is passed onto an intermediate layer or hidden layers which is then fed into the output layer to produce a linear result. Modelling will be done in two phases. First, tuning the lags then tuning the hidden layers. As ANN models are susceptible to random initial weights, multiple random seed environments are used and the mean of each metric is calculated to provide the most accurate comparison.

Lags chosen to be tested will be based on previous models as testing them one-by-one is time and resource consuming. In addition, up to two hidden layers with three different neuron numbers, {10, 20, 30}, will be used for the same reason. In total, ten different ANN models with different lag and hidden layer configuration will be tested and compared to each other. Table VII records the performance of these ten configurations.

TABLE VII. ANN Models Performance

Lag	Hidden Layer	RMSE	MAE	MAPE (%)
1	{30}	56.436	45.028	0.6281
3	{20}	88.609	70.479	0.9850
1, 3	{10}	55.610	44.205	0.6162
1, 3	{10, 20}	55.514	44.115	0.6149
3	{20, 10}	88.348	69.708	0.9744
1, 2, 3	{30, 20}	55.822	44.402	0.6193
1, 2, 3	{10}	56.501	44.663	0.6224
3, 7	{20, 10}	88.618	71.456	0.9946
7	{10, 30}	115.054	91.667	1.2818
1, 7	{10}	57.330	46.210	0.6437

From the table, the performance of the models largely depends on the lags used. As seen in the results, models that include lag 1 tend to perform better than models which don't. These models achieved relatively low average RMSE, MAE and MAPE values when compared with the other models. The complexity of the hidden layer also has a small impact towards the performance of the models. Models with higher hidden layer complexity tend to perform better than more simple ones, though that may not always be the case.

The best performing model is ANN with Lag(1,3) and hidden layers {10, 20} as it achieved the lowest average RMSE, MAE and MAPE values valuing at 55.514, 44.115 and 0.6149% respectively.

4.7. Model Comparison

The best performance from each forecasting method will be compared with each other to determine the single best model using the test set. Table X records the performance of these models.

TABLE X. Forecasting Models Performance

Model	RMSE	MAE	MAPE (%)
Naive with Drift h = 112	135.308	103.035	1.446
SES(0.3)	154.408	115.498	1.629
DES(0.1, 0.05)	156.138	118.946	1.675
ARIMA(2, 1, 2)	165.317	140.875	1.963
ANN Lag(1,3) {10, 20}	55.514	44.115	0.6149

From the results, traditional models perform worse than the benchmark Naive with Drift model. This shows that Naive with Drift is the best-performing traditional model, outperforming both exponential smoothing models and ARIMA with the lowest RMSE, MAE, and MAPE values amongst them. This indicates that Naive with Drift is able to capture the non-linear and random walk patterns of the data better than other traditional models. The worst-performing traditional model is ARIMA(2, 1, 2).

On the other hand, ANN greatly outperforms the traditional models presented. With a very low RMSE and MAE in the double digits and MAPE values in the decimals, ANN is able to capture the non-linear relationships within the data. The best model to be used is ANN using lags 1 and 3 with 2 hidden layers that have 10 and 20 neurons, respectively.

Overall, the best-performing model from the results is ANN with Lag(1,3) and hidden layer {10, 20} as it achieved the lowest RMSE, MAE & MAPE value out of all of the models. As seen before in the naive models, Naive with Drift using a shorter forecasting period outperforms every other model here, even ANN. However, the main drawback of the Naive with Drift model is that it can only forecast short periods. Meanwhile, ANN is better for long-term forecasting. Therefore, ANN is more suitable for long-term forecasting, while Naive with Drift is more suitable for short-term forecasting for 14 periods.

5. CONCLUSION

This study developed a quantitative framework to evaluate and compare traditional statistical models and machine learning models for forecasting daily movements in the Jakarta Composite Index (JCI). This study evaluates the performance and weaknesses of various forecasting methods with the same dataset.

The results show that traditional linear models are difficult to use on highly volatile financial data. Even though Naive with Drift is acceptable as a standard, its performance diminishes when rapid changes are happening in the market, which leads to larger errors over longer periods when forecasting.

On the other hand, artificial neural networks work more efficiently. This is due to the fact that ANN is able to capture data that have a more complex pattern, giving it the ability to adapt better to market behaviors changing. Moreover, using random seeds makes the results more stable in the end.

All in all, ANN produces better results when compared to standard models in predicting stock (JCI) data that are out-of-sample. This is even more true for longer prediction periods, which in turn helps investors and market analysts to make better decisions and manage risk in the Indonesian stock market.

REFERENCES

[1] Y. C. Tjen, "Peramalan IHSG dengan Metode ARIMA-GARCH dan LSTM Pada Periode Sebelum, Masa dan Sesudah Covid-19," Master's thesis, Universitas Islam Indonesia, Yogyakarta, 2024.

[2] N. B. Syahaza, M. R. Fajri, & S. Wahyuni, "Forecasting the Indonesian Stock Market Index Using ARIMA–GARCH Models with a Rolling Window Approach," *ResearchGate Preprints*, pp. 1-12, Feb. 2026.

[3] R. S. Hyndman and G. Athanasopoulos, "Chapter 5: The forecasting toolbox," in *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

[4] K. D. Makatjane & E. P. Molefe, "Predicting regime shifts in Johannesburg stock exchange allshare index (JSE-ALSI): A Markov-switching approach," *Eurasian Journal of Economics and Finance*, vol. 8, no. 2, pp. 95-103, 2021.

[5] R. Nindiyani, "Perbandingan Metode Artificial Neural Network dan Gated Recurrent Unit Untuk Memprediksi Indeks Harga Saham Gabungan (IHSG) Periode 2014-2024," Undergraduate thesis, Universitas Lampung, Bandar Lampung, 2025.

[6] R. S. Hyndman & G. Athanasopoulos, "Chapter 9: ARIMA models," in *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

[7] B. Hamzah & J. Hartono, "Analisis Prediksi Indeks Harga Saham Gabungan (IHSG) Menggunakan Metode Nonlinear Auto-Regressive Exogenous Model (NARX) Neural Network dan Metode Long Short-Term Memory (LSTM)," Master's thesis, Universitas Gadjah Mada, Yogyakarta, 2023.

[8] M. Konur, M. Gocken, & A. T. Dosdogru, "Stock Price Prediction Using Deep Learning Algorithms Based on Technical Indicators," *Journal of Risk and Financial Management*, vol. 15, no. 10, pp. 412-425, 2022.

[9] M. A. Majid, P. D. Saputri, & Soehardjoepri, "Stock Market Index Prediction using Bi-directional Long Short-Term Memory," *Journal of Applied Informatics and Computing (JAIC)*, vol. 8, no. 1, pp. 55-61, 2024.

[10] A. T. Biva, A. S. Hossain, & M. R. Islam, "Benchmarking Hybrid ANN-LSTM and Physics-Informed Neural Networks for Forecasting Stock Market Prices," *International Journal of Computer Applications*, vol. 186, no. 74, pp. 11-19, 2025.

[11] M. Hagag, "Assessing Machine Learning Linear Models to Predict Egyptian Stock Market Prices," *International Journal of Computer Applications*, vol. 184, no. 43, pp. 30-37, 2023.

[12] D. Samarasuriya, "Recurrent Neural Network for Stock Market Forecasting using Long Short-Term Memory," *International Journal of Computer Applications*, vol. 186, no. 10, pp. 5-13, 2024.

[13] M. Bagmar, "StockSense: AI-Powered Prediction with Real-Time News Intelligence," *International Journal of Computer Applications*, vol. 187, no. 94, pp. 43-50, 2026.

[14] K. K. Ajoku, O. C. Nwokonkwo, & A. M. John-Otumu, "A Model for Stock Market Value Forecasting using Ensemble Artificial Neural Network," *International Journal of Computer Applications*, vol. 182, no. 48, pp. 1-8, 2021.

[15] A. S. Pinto, "Predicting stock price direction using machine learning models," Master's thesis, Business School, Iscte Instituto Universitário de Lisboa, Lisbon, 2023.

[16] C. Zhang, "Movement Prediction-Adjusted Naive Forecast: Is the Naive Baseline Unbeatable in Financial Time Series Forecasting?," arXiv preprint arXiv:2406.14469, revised Oct. 2025.

[17] B. G. Malkiel, *A Random Walk Down Wall Street*, 13th ed. New York, NY, USA: W. W. Norton & Company, 2023.

[18] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, & G. M. Ljung, Time Series Analysis: Forecasting and Control, 5th ed. Hoboken, NJ, USA: John Wiley & Sons, 2015, ch. 6, pp. 183.

[19] R. S. Hyndman & G. Athanasopoulos, "Chapter 12: Advanced Forecasting Methods," in *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.